

COMP6841 – Final project report

Presented to Instructor Tri Minh Triet and Firat Oncel (TA)

May 1st 2026

Abstract

This study develops a Deep Learning extension of the OhmNet framework to improve the prediction of tissue-specific protein-protein interactions (PPIs). While traditional models often treat biological tissues as independent entities, this research implements a hierarchy-aware architecture that enforces the "True Path Rule"—a fundamental biological principle requiring that interactions in specialized child tissues be consistently represented in their parent organs. This clinical constraint is vital for pharmacovigilance and drug-discovery applications, where logical consistency across anatomical structures is necessary for model reliability.

The methodology utilizes a late-fusion Graph Attention Network (GAT-MLP) integrated with 256-dimensional Node2Vec tissue embeddings to capture the complexity of the BRENDA Tissue Ontology. A primary focus of this work was the development of a dynamic penalty scheduling system to manage the "objective competition" between classification accuracy and hierarchical consistency. Experimental results on an NVIDIA RTX 3090 Ti demonstrate that an unconstrained penalty leads to catastrophic loss divergence, whereas a capped, rule-based multiplier (λ) allows the model to satisfy biological constraints without sacrificing predictive power. The final architecture achieves a Macro-AUROC of 0.995 and reduces hierarchical violations to less than 1%, offering a robust framework for consistent, large-scale biological network analysis.

Introduction

The purpose of this project was to build a Deep Learning extension of a study on prediction of tissue-specific protein-protein interactions (PPIs)[1] by building a tissue hierarchy-aware, edge prediction, late-fusion GAT-MLP model. The original work, the OhmNet framework, is a multi-label classification task that introduced a regularization method that factored ontological hierarchy. Every protein in the network was assigned zero, one or more tissue-specific cellular functions for the classifier to be trained on features from other tissues. It also featured a transfer learning component where they predicted "cellular functions for a protein in one tissue based on classifiers trained on features from other tissues"[1]. Finally, they performed a network-embedding of tissues for visualization in 2D space.

The approach of this work is to perform a similar set of prediction tasks but this time using semi-supervised learning and high-dimensional embeddings for tissue hierarchy. A late-fusion[2] GAT-MLP model was used for training assisted by a high-dimensional hierarchical tissue embedding system. The loss function used is BCELoss() with an added modular regularization component called the hierarchical penalty ($\lambda * \text{penalty}$). While the original OhmNet framework generated embeddings for 219 tissues, it utilized a limited training set of 144 tissues, evaluating performance specifically on 107 leaf nodes. In this work, we implemented bottom-up label propagation based on the True Path Rule[3] to address the 75 parent nodes absent from the original training data. Label propagation was felt necessary for two reasons: First, without representing these 75 parent nodes, the GAT-MLP would be unable to map the hierarchical relationships captured by the high-dimensional embeddings to actual protein interaction patterns. Propagation ensures the model learns the inheritance of biological function across all 219 nodes in the embedding space. Second, since the custom loss function explicitly penalizes hierarchical violations (HV), the presence of 'hidden' parent interactions (labeled as 0 in the original sparse data despite children being 1) would have artificially inflated the penalty term. Propagating the labels prevents these 'false violations' from dominating the gradient, thereby stabilizing the training dynamics and allowing the BCE loss to converge meaningfully. With label propagation, it was possible to reduce the un-penalized hierarchical violation rate from around 64% to a more realistic 22%, a better starting point.

The primary objective of this work is to evaluate whether modern GNN architectures can surpass the original OhmNet benchmark of 0.756 AUROC across 107 leaf-node tissues. By transitioning from traditional regularization to a deep-learning-based attention model, the hope is to show that Deep Learning architectures can be applied to traditional problems to increase the reliability of the results through the analysis of relevant metrics all while demonstrating biological consistency through hierarchical tissue violations rates.

It is important to acknowledge that the significant performance gains observed in this work—including an AUROC of 0.9951—cannot be viewed as a direct longitudinal comparison to the original OhmNet benchmark. The introduction of bottom-up label propagation and the Dynamic Hierarchical Penalty fundamentally shifts the problem from the sparse, unconstrained optimization of the 2017 study to a biologically-consistent, semi-supervised framework.

While the original OhmNet framework utilized an indirect maximization algorithm based on manifold regularization to encourage hierarchical similarity, this work introduces the penalty as a novel Ontological Evaluator. By moving from a hidden regularization term to an explicit penalty, a transparent metric for 'Biological Consistency' that directly quantifies the model's adherence to biological logic in real-time is provided, a dimension of validation that was not directly measurable in the original study.

However, as a Deep Learning evolution of the OhmNet project, I believe these results (AUROC, AUPRC, Macro-F1, and Micro-F1) remain entirely valid. Rather than merely optimizing for a specific metric, this work demonstrates that by aligning modern GNN architectures with the logical constraints of the True Path Rule, we can achieve higher predictive reliability without sacrificing biological integrity. The results presented here represent the upper bound of performance attainable when structural anatomical priors are successfully integrated into the learning process.

Methodology

Architectural components – a synthesis

A Directed Acyclic Graph (DAG) of the 219 tissues used in the study was constructed using NetworkX[4], [5], [6], [7] and embeddings were generated with Node2Vec[8], [9]. With assistance in the labeling logic, bottom-up label propagation was performed to adhere to the True Path Rule which increased label density from 23.8% to 38.3% while maintaining the same 30.5% label density for the 107 leaf tissues. The label propagation was verified to involve the complete set of 219 tissues, an increase from the original 144 and the 75 tissues to which labels were propagated were identified. A multi-hot tissue adjacency matrix based on 3.6M PPIs (144 tissues of which 107 are leaf nodes that will be used for testing just like in the original work) was generated using the OhmNet PPI dataset[10].

The model originally utilized two GAT layers with 16-1 heads to learn contextual protein representations, using ELU activation in the hidden layer[11]. With further analysis of the results and state of the art GAT models, a 3-layer 8-8-1 GAT was adopted to better represent PPI and neighborhood features. These learned features are then concatenated with the pre-trained Node2Vec tissue embeddings in a Late Fusion layer. This combined representation is processed by a 1-layer MLP (ReLU-activated) whose final linear outputs are passed through a Sigmoid function to generate multi-label interaction probabilities. The model is optimized using binary cross-entropy (BCE) augmented by a custom hierarchical violation penalty to ensure biological consistency across the tissue ontology.

Hyperparameters

The final hyperparameters chosen for the project were as follows:

Category	Hyperparameter	Value
Tissue Embeddings	Node2Vec Dimensions	64
	Walk Length	10
	Context Size	10
	Walks per node	10
	Number of Negative Samples	5
	Sparsity	True
	Optimizer	SparseAdam
	Learning Rate	0.01
	Batch Size (N = 219)	32
	Node2Vec Epochs	1000
GAT architecture	GAT v2 Conv layers	3
	Attention Heads	8
	GAT Hidden Channels	256
	Dropout rate	0.3
	Activation Function (GAT layer 1)	Elu (F.elu)
MLP Classifier	MLP Layer	1
	MLP Hidden Channels	1024
	Activation Function (MLP layer 1)	ReLu (nn.ReLu)
Training Configuration	Batch Size (N = 70 343)	1024
	Adaptive Lambda Penalty (initial)	200
	Lambda Penalty limit (cap)	300
	Optimizer	Adam
	Learning Rate	0.0005
	Patience (early stopping on val losses)	50

Data acquisition and pre-processing

The raw OhmNet dataset consists of approximately 3.6 million tissue-specific protein-protein interactions (PPIs) across 144 tissues, supported by a tissue hierarchy file containing 219 tissue nodes. For this study, these were collapsed into a 219-column multi-label interaction matrix featuring 70,338 unique protein pairs. Initially, the dataset exhibited significant sparsity, with a label density of approximately 23.8%.

Exploratory analysis identified critical hierarchical inconsistencies: many interactions were annotated in specialized child tissues (e.g., left ventricle) but omitted in their respective parent nodes (e.g., heart). Within a biological ontology, this represents a logical impossibility, as a specialized interaction must, by definition, be present within the broader anatomical structure.

To ensure the model learned from a biologically consistent ground truth, a label propagation algorithm was implemented. This algorithm enforces the "True Path Rule"[3]: if an interaction is active in a child node, it is

automatically propagated to all ancestor nodes in the hierarchy. This correction resolved logical violations and increased the training label density to 38.3%, providing a more robust signal for the GAT-MLP model.

It is important to note that this augmentation primarily addressed the 75 parent nodes missing from the original PPI dataset, ensuring consistency between the tissue embeddings and the interaction matrix. During inference, this process did not affect the leaf-node metrics, as the label density for evaluation remained at 30.57%.

```
Propagating labels through hierarchy...
--- Global Density (All 219 Nodes) ---
Original label density: 23.8026%
Propagated label density: 38.3389%

--- Tissue Coverage ---
Tissues represented in raw PPI: 144 / 219 (65.8%)
Tissues active after Propagation: 219 / 219 (100.0%)

--- Specific Metrics ---
Propagated leaf node density: 30.5682%
```

Figure 1 - Label density pre- and post-propagation

Node2Vec, tissue embeddings

To generate the tissue embeddings, Node2Vec dimensions were initially set to 64. However, through empirical testing, a dimension of 256 was ultimately selected. This choice was driven by a need for higher representational capacity: with 219 tissues in the ontology, a 64-dimensional space risked collapsing distinct structural relationships. A 256-dimensional embedding provides the 'mathematical space' necessary to encode the biological complexity of the hierarchy without creating significant computational overhead or exceeding VRAM limits. Observations indicated that while Node2Vec losses remained relatively stable across higher dimensions but 256 provided a sufficient balance between training efficiency and feature expression.

Similarly, the parameters for the random walks, that is `walk_length=20`, `context_size=10`, and `walks_per_node=10`, were empirically validated. Given the depth of the BRENDA ontology, these settings might initially appear to be an overestimation, potentially causing walks to traverse entire sub-sections of the tree. However, a trial performed by reducing these settings (`walk_length=10` and `context_size=5`) resulted in consistently slightly higher losses. The higher walk parameters ensure that even distant parent-child dependencies are captured, which is critical for the later enforcement of hierarchical consistency during training.



Figure 2 - BTO tree of the 219 tissues in the tissue.hierarchy file, the 107 leaf tissues are colored in blue

To optimize the training of the 256-dimensional embeddings, we specified `sparse=True` and utilized the SparseAdam[12] optimizer. This configuration ensures that during each step of the random walk, only the gradients for the sampled nodes are calculated and updated, drastically reducing the computational overhead and accelerating convergence by avoiding redundant updates to the inactive portions of the tissue hierarchy.

Xavier initialization was utilized for the 256-dimensional tissue embeddings out of necessity. During testing initial losses for randomly initialized weights were around 4.7 and even 1000 epochs wasn't sufficient to reach convergence. After researching the root of the problem it was apparent that a form of initialization was

necessary and Xavier was chosen simply because of its prior use in graph-type problems[11]. Following this, the initial losses were below 0.8 and they consistently reach approximately 0.75 after 1000 epochs.

Another important note: research had led to the testing of a 10d poincaré embedding system because they are said to be ideal for trees/hierarchies[13] but it is possible that 10d representations were too densely populated for the 219 tissues which led to important irrecoverable losses. The losses were unfortunately not documented but the code for the poincaré embeddings remains if testing is needed.

The loss function used to train the embeddings is CCE.

Of important note, the weights of the tissue embeddings are frozen with `detach()` at the end to prevent backpropagation from reaching the learned Node2Vec embeddings during training.

Neural network architecture

GAT

The GATv2 component consists of 3 layers with residual connections. The first layer (`gat1`) maps the initial protein embeddings to 256-dimensional hidden features across 8 attention heads. These multi-head outputs are concatenated and ELU-activated[11] to produce a 2048-feature intermediate representation. Those representations are fed to a second 256 x 8 GATv2 layer yielding another 2048 vector per PPI. The third layer aggregates these features into a final 256-dimensional embedding per protein.

MLP

For the classification task, a Late Fusion approach is employed. The MLP receives a 768-dimensional triplet input consisting of the concatenated embeddings of Protein A, Protein B, and the Node2Vec tissue embedding. This input passes through a 1024-node hidden layer with ReLU activation and dropout. The final linear layer reduces the feature space to a single scalar per tissue-pair, which is transformed via a Sigmoid function to provide independent probabilities for multi-label tissue prediction.

Dropout

A hyperparameter search was performed on dropout (0, 0.3, 0.4, 0.5) with the hierarchical penalty portion of the loss function disabled. The default of 0.5 was first tested to yield satisfactory results but tests at lower dropout yielded even better results with 0.0 performing the best. Once the hierarchical penalty was introduced, zero dropout still performed the best in terms of statistical metrics but dramatically worse in terms of hierarchical violations (9% vs around 0.5% with dropout enabled). There are further reasons to choose dropout over no dropout with the hierarchical penalty enabled but this will be explored when discussing the custom loss function.

As far as dropout rate is concerned, there is further evidence that suggests that dropout rates of 0.3-0.4 are ideal for GATs in [14] where they proposed implementation of a variable dropout rate through training. Their preliminary results however suggest 0.3-0.4 rates are optimal for GATs.

Important technical note, in GATs, there are 2 types of dropout namely feature dropout and attention dropout. Feature dropout ensures the model doesn't co-adapt to its own feature dimensions whereas attention dropout will calculate the weight of the attention coefficient then randomly dropout a percentage of attention weights and renormalize the remaining attention weights so they sum up to 1, ensuring the model doesn't co-adapt to specific neighboring protein nodes. For this project, attention dropout was used exclusively as it is the type of dropout embedded in Pytorch Geometric's `GATv2Conv()` function.

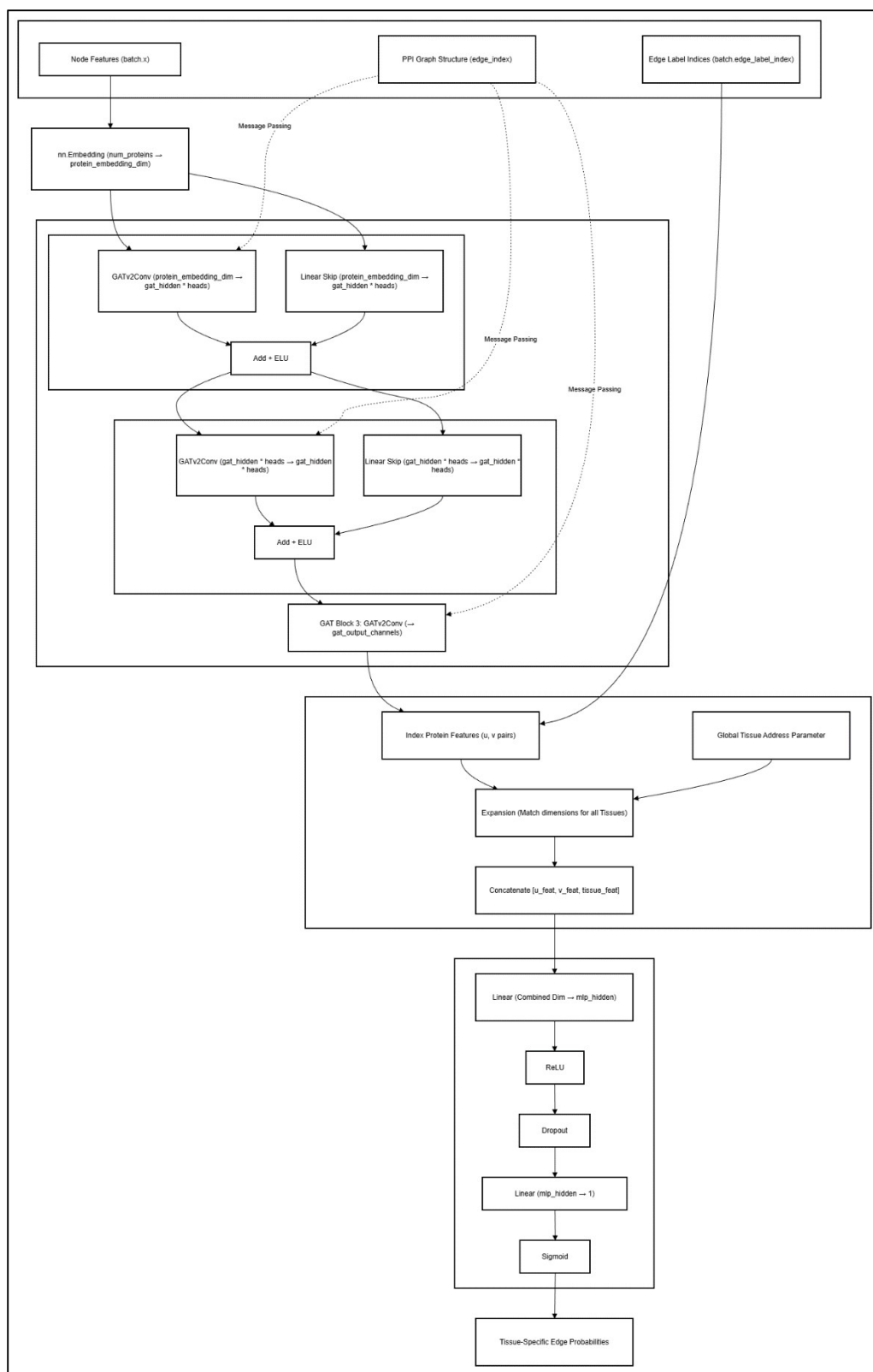


Figure 3 - Deep Learning architecture during training

Custom loss function

A custom loss function was designed in stages to add hierarchical violation penalties. The reasoning is that there should be a penalty if the predicted interaction in a child node is higher than the predicted interaction in the parent node. The equation below describes the concept:

$$Loss = BCE(y, y_{hat}) + \lambda \sum (P_{child} - P_{parent}) \text{ where } P_{child} > P_{parent}$$

The expanded idea behind the hierarchical penalty loss portion is the following:

$$L_{total} = L_{CE} + \lambda \cdot L_{hierarchy}$$
$$\text{where } L_{hierarchy} = \frac{1}{batch_size} \sum_{(child, parent) \in \epsilon_{Hierarchy}} \max(0, y_{hat_{child}} - y_{hat_{parent}}) + margin$$

This introduces an additional hyperparameter λ which in the context of this project is named lambda_penalty (λ). A hyperparameter search was performed with BCELoss() as the baseline and $\lambda = 0.1, 0.5$. At 0.5, the metrics (AUROC, AUPRC, Macro-F1 score) were good but through an assisted design of a hierarchy violation rate (HV/HVR) function[15], I discovered that with simple BCELoss() and low lambda penalties yielded high tissue hierarchy violations.

Without proper penalty scaling ($\lambda = 0.5, 1.0$), the model was good at predicting what interactions happened in which tissues but wasn't observing the biological constraints. My work would've been a lot less complicated had I settled for good AUROC and AUPRC but now a different approach was necessary. With consultation, it was determined that a starting lambda of 1 with a scheduled multiplier of 1.2 if hierarchical violations were not decreasing was necessary. A test trial led to early stopping being triggered at epoch 643 and a final lambda 114.5 with respectable AUROC and AUPRC.

Seeing that the penalty had increased about 30 times in a single trial led me to increase the initial lambda to 50 and see the results. This trial led to a final lambda of 642.0, a test hierarchy violation of 0.04%, respectable AUROC and AUPRC but an explosion of the BCELoss() in the final stages of training at around epoch 180.

Given that this methodology led to the model always trying to improve hierarchical violations at the expense of BCELoss(), the idea came to simply gate the tissue violation percentage to 1% (a seemingly arbitrary but acceptable hierarchy violation rate) and to schedule penalty increases only when hierarchy violation rate went above the threshold. It is then that the learning rate was decreased to 0.0005 from the value of 0.001 I had maintained for a dozen or so trials. The final results are featured in the appropriate section.

After much experimentation, the hierarchy penalty λ was re-designed into a capped, dynamically scheduled, rule-based multiplier acting as a conditional logic gate. If the hierarchical violation rate exceeded 1%, λ , which is set to a base value of 200, would increase by 10% up to a maximum of 300. Those values were empirically selected after many trials starting with a base uncapped lambda of 1 and increasing by 20% every time the hierarchical violation rate was above 1%. At less than 200, the overall penalty is too low and the model only pays attention to BCE only and at more than 300, the model pays too much attention to the hierarchical penalty and the BCE eventually skyrockets.

A confidence margin ($m = 0.1$) was introduced in the hierarchical penalty to further guide the model into making more confident predictions about parent and node interactions. This margin requires the model to be at least

10% more certain that an interaction occurred in the parent node vs the child node. Predictions failing to maintain this probability gap are penalized, even if they are technically consistent with the hierarchy.

Inference

During inference, a pair of proteins is presented to the model. Using a LinkNeighborLoader, a local subgraph is sampled around these proteins to provide the GAT with functional context. The proteins are transformed through successive GAT layers into deep, structurally-aware embeddings. These two protein embeddings are then concatenated with a Node2Vec-derived tissue embedding for each unique 219 tissues to form a series of combined feature vectors. These vectors are passed through a fully connected layer (MLP) to generate logits, which are then converted into probabilities via a sigmoid function. If the probability for a given tissue is above 0.5, the interaction is predicted to exist within that anatomical context.

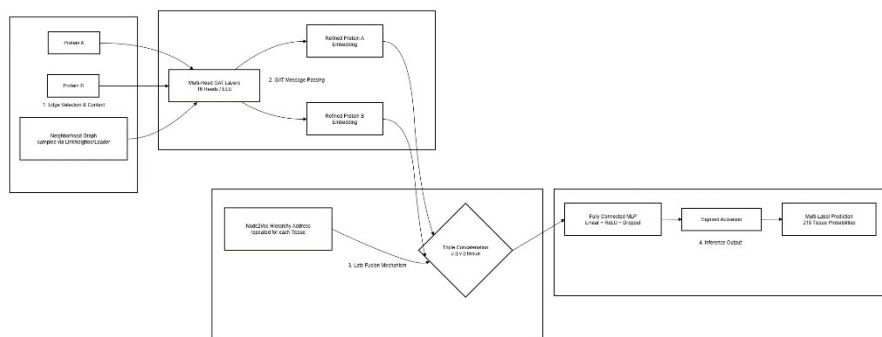


Figure 4 - Deep Learning process during Inference

Technical specifications

All model training and high-dimensional embedding generations were performed on a local workstation equipped with an AMD Ryzen 7 5800X CPU and an NVIDIA GeForce RTX 3090 Ti (24GB VRAM). The substantial VRAM capacity was critical for accommodating the large-scale graph adjacency matrices and the high-dimensional (256D) Node2Vec embeddings. The software stack was built on Fedora Linux (or Windows, depending on which you're focusing on for the report) utilizing PyTorch 2.5.1 and PyTorch Geometric 2.7.0 for graph neural network operations. NetworkX was employed for hierarchical graph manipulations, and Weights & Biases (W&B) was integrated for real-time telemetry, specifically to monitor GPU thermal performance, losses and the stability of the dynamic λ scheduling.

Results

Architectural design: Width vs Depth

Table 1 lists the metrics of the initial 16-1 GAT architecture (“wide” GAT) vs the adopted 8-8-1 GAT architecture (“Deep” GAT. Figures 5 and 6 are the loss curves and metrics of the “wide” GAT and Figures 7 and 8 repeat the same but for the “Deep” GAT. A number of factors are important to consider when looking at the result and they are addressed in the Discussion.

Table 1 - Metric comparing a wide vs a deep GAT architecture - Dropout 0.3, custom loss function*

Metric	“Wide” GAT	“Deep” GAT
AUROC	0.9751	0.9945
AUPRC	0.8957	0.9717
Macro-F1	0.8295	0.9282
Micro-F1	Undocumented at the time	0.9712

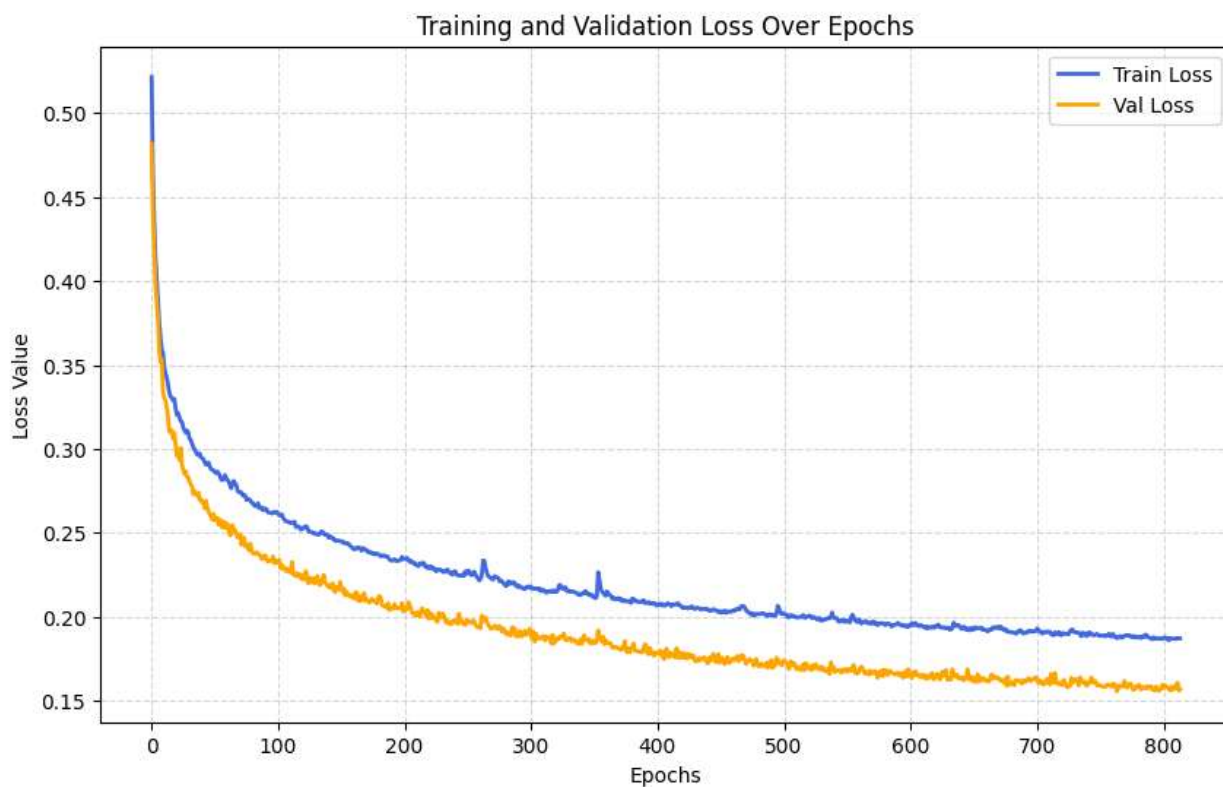


Figure 5 - Loss curves with dropout = 0.3, lambda = 0.5 for the 16-heads, 2-layer GAT

```
Calculating metrics for 107 leaf tissues...
--- Leaf Tissue Results ---
Test Leaf AUROC: 0.9751
Macro-AUPRC: 0.8957
Macro-F1 Score: 0.8295
Based on test AUROC score, the project was a success for achieving an AUROC score significantly higher than 0.756!
(Based on 107 tissues that had valid test data)
```

Figure 6 - Metrics with dropout = 0.3, lambda = 0.5 for the 16-heads, 2-layer GAT

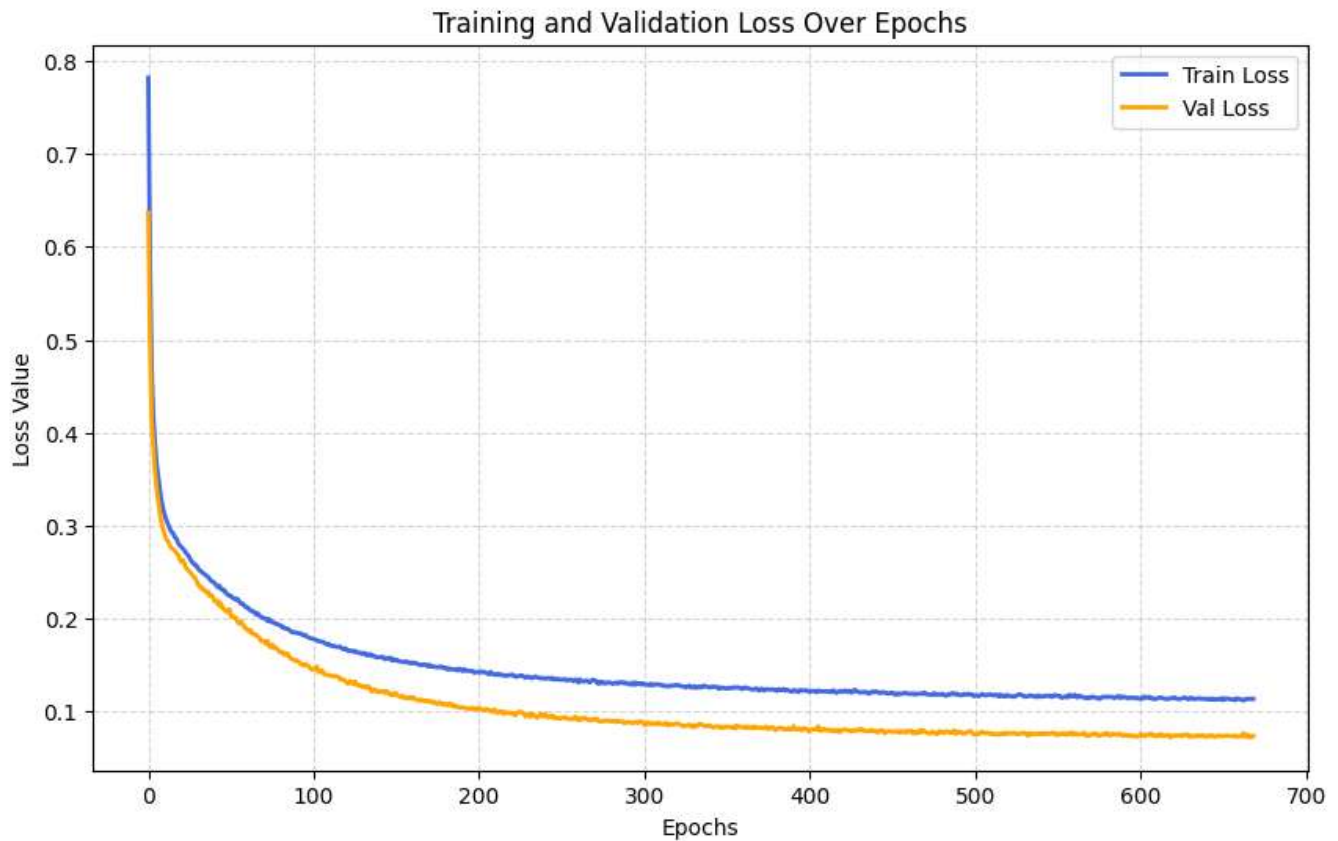


Figure 7 - Loss curves for Dropout = 0.3, λ initial = 200, custom loss function for the 8-heads, 3-layer GAT

```
Collecting predictions for 14068 test edges...  
--- Final Evaluation Results (107 leaf tissues) ---  
Macro-AUROC: 0.9945  
Macro-AUPRC: 0.9717  
Macro-F1 (using val-set threshold 0.40): 0.9282
```

```
Test Micro-F1 Score: 0.9712
```

Figure 8 - Metrics for Dropout = 0.3, λ initial = 200, custom loss function for the 8-heads, 3-layer GAT

Custom Loss function – λ and HV

Figure 9, 10 and 11 is the experiment on the 3-layer GAT with an unbound λ and no set tolerance for hierarchical violations. Initial $\lambda = 1$, increased by a factor of 1.2 as long as the model detects hierarchical violations. Final λ was around 643 and we can see the losses increase dramatically around epoch 170 and a slight recovery is seen but not enough to prevent the early stopping mechanism from halting the experiment.

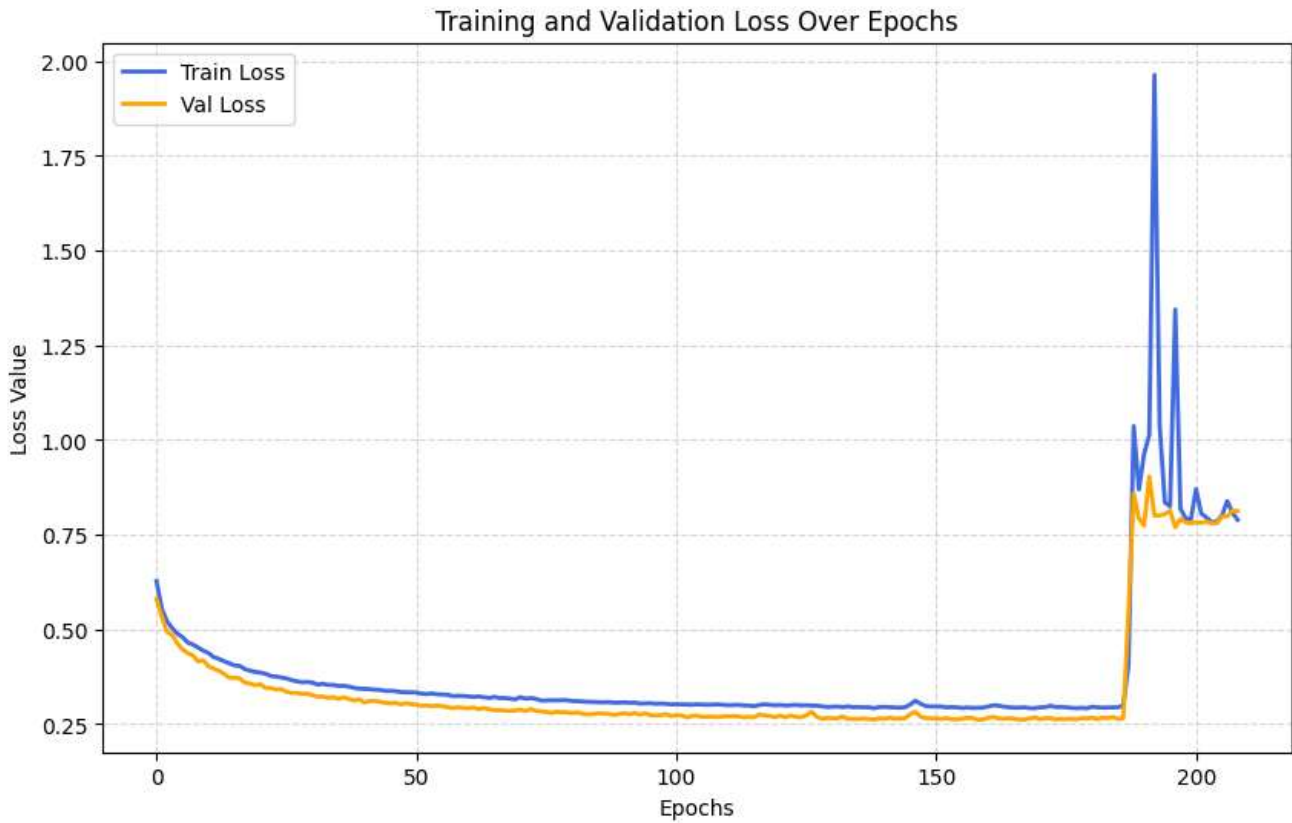


Figure 9 - Loss curve demonstrating the effect of uncapped penalties. Dropout = 0.3, λ uncapped, no hierarchical penalty constraints.

```
Calculating metrics for 107 leaf tissues...
--- Leaf Tissue Results ---
Test Leaf AUROC: 0.9326
Macro-AUPRC: 0.7796
Macro-F1 Score: 0.6605
Based on test AUROC score, the project was a success for achieving an AUROC score significantly higher than 0.756!
(Based on 107 tissues that had valid test data)
```

Figure 10 - Early metrics with custom loss, uncapped λ and unconstrained model on hierarchical penalty

```
--- Model Hierarchical Performance (Test Set) ---
Violation Rate: 0.04%
Violations: 1,284 / 3,066,824
```

Figure 11 - Hierarchical violation rate when model is unconstrained on violation rate

The effect of dropout on convergence

During hyperparameter search for dropout and tests for the impact of the custom loss function, some tests were run with dropout = 0, 0.3, 0.4 and 0.5 without the custom loss function. Figure 12 shows the loss curves with dropout = 0 and Figure 13 with dropout = 0.3. Notice the validation losses being higher than training losses in Figure 12 and lower than training losses in Figure 13.

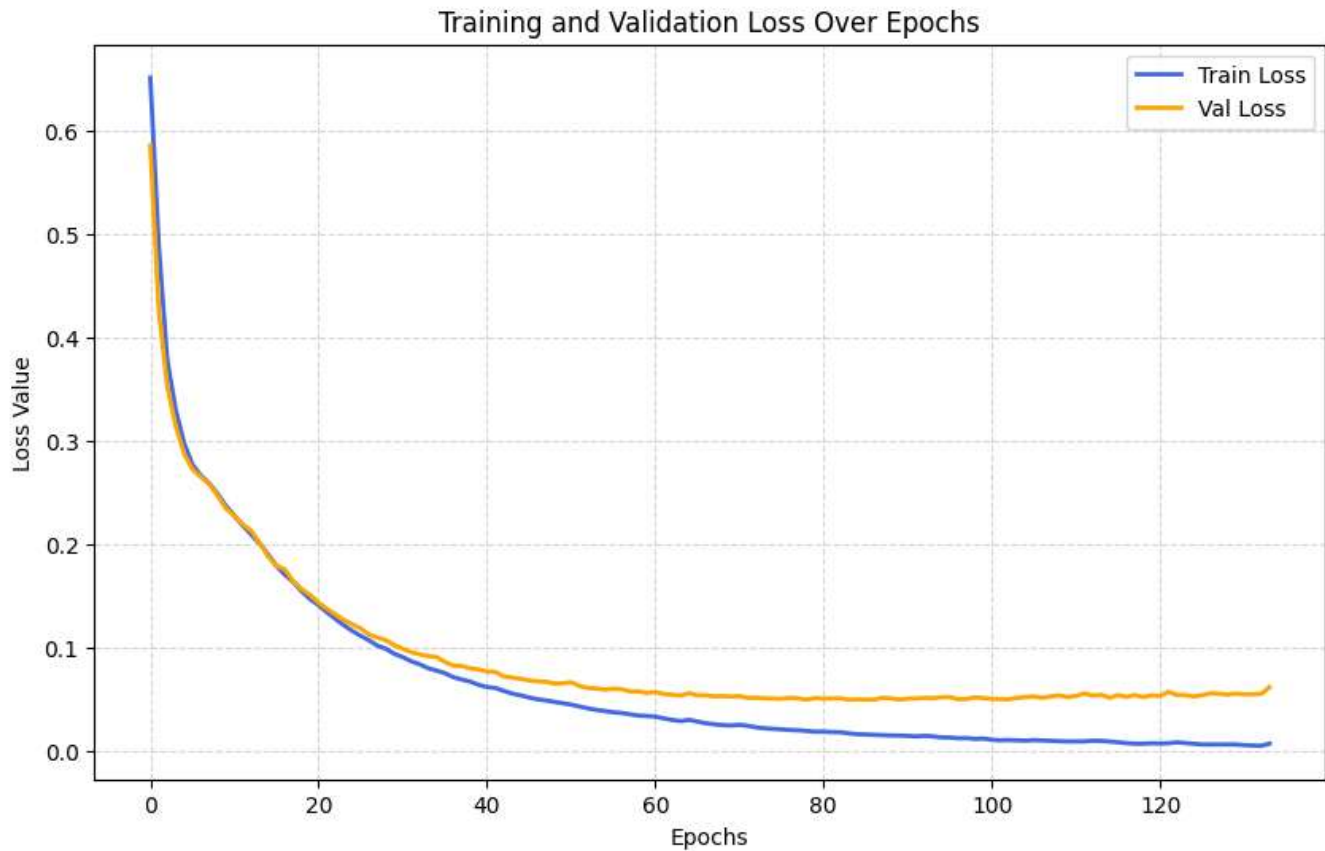


Figure 12 - Loss curves with dropout = 0 and BCELoss() as the single loss component

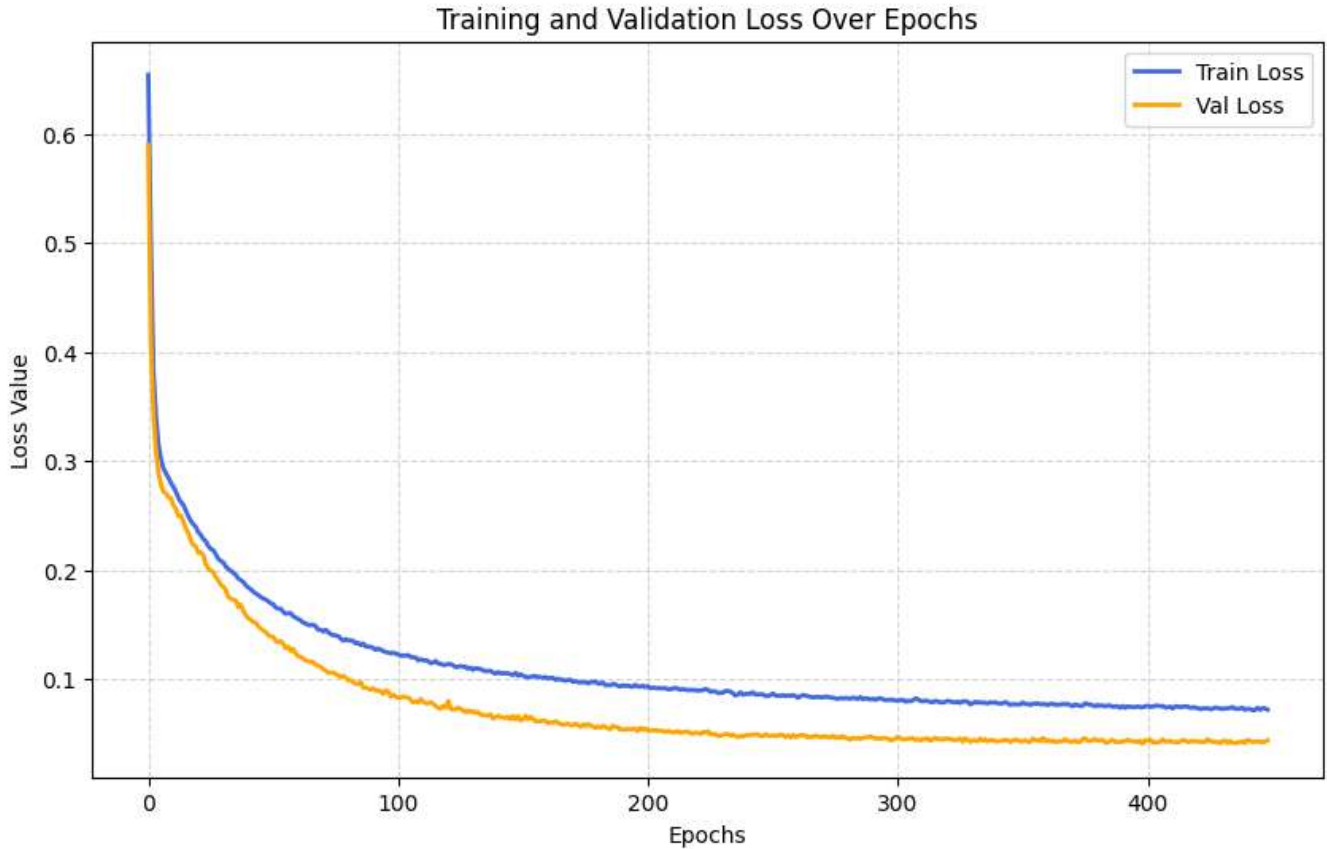


Figure 13 - Loss curves with dropout = 0.3 and BCELoss() as the only loss component

Final results

Based on empirical data gathered throughout the ablations studies, the following final hyperparameters were selected for the production model: 3-layer 8-8-1 heads GAT with a Dropout rate of 0.3 and learning rate of 5×10^{-4} . For the custom loss function: initial $\lambda = 200$, λ growth factor = 1.1, λ ceiling = 300, hierarchical violation rate tolerance = 1%.

It is important to distinguish between structural hyperparameters such as the choice of network depth (deep vs. wide) and training hyperparameters like the λ scheduling. Figure 14 illustrates the stabilized loss curves achieved with this curated configuration. Table 2 summarizes baseline performance using BCELoss() exclusively, while Table 3 presents the final metrics achieved with the integration of the hierarchical penalty, demonstrating the model's ability to maintain high predictive accuracy alongside ontological consistency.

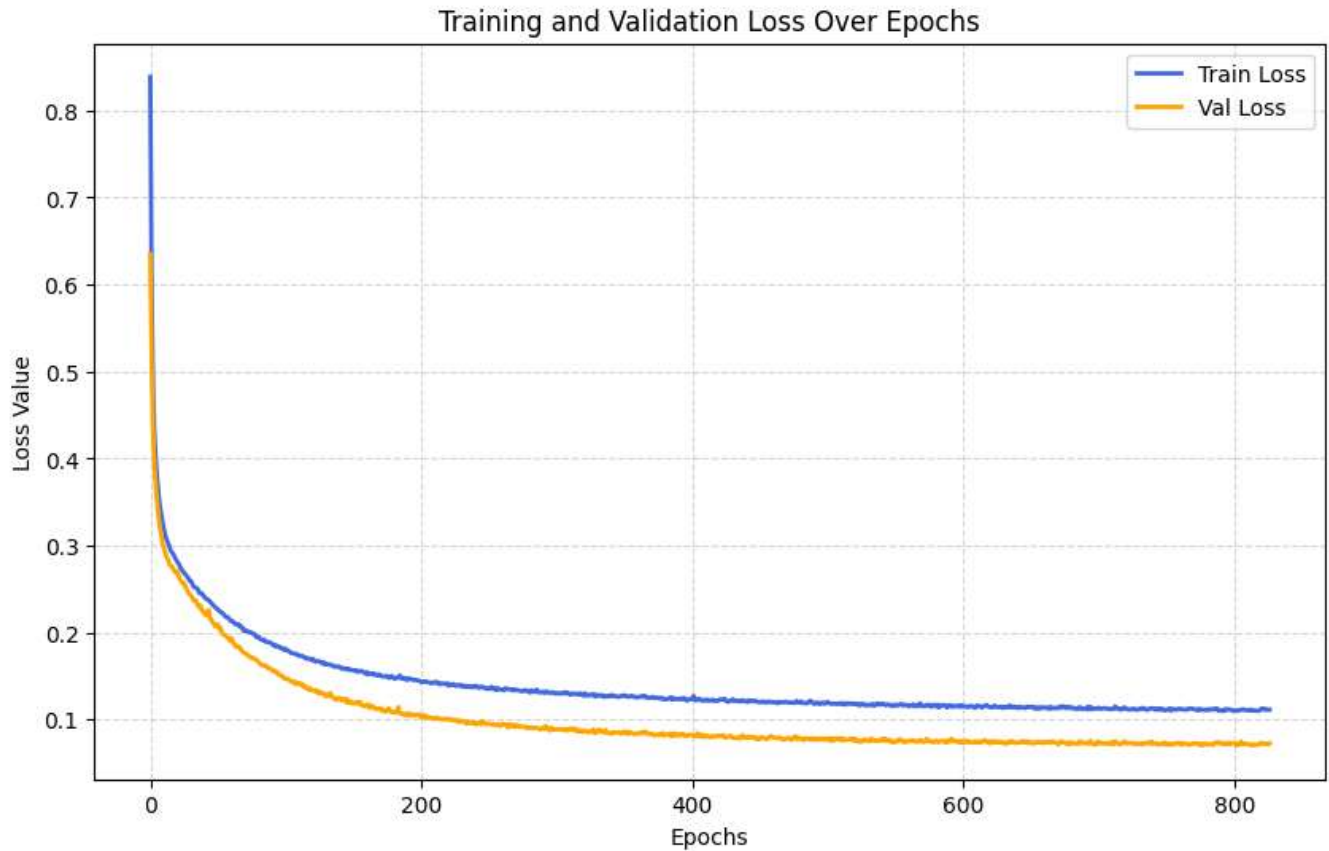


Figure 14 - Loss curve for final hyperparameter run with dropout = 0.3 and custom loss function

Table 2 - Dropout rate hyperparameter search with BCELoss() as the single loss function component

Dropout rate	AUROC	AUPRC	Macro-F1	Micro-F1	Threshold	Violation rate
0.0	0.9977	0.9915	0.9714	0.9881	0.45	22.68%
0.3	0.9970	0.9880	0.9636	0.9852	0.40	21.16%
0.4	0.9948	0.9752	0.9316	Undocumented	0.35	20.25%
0.5	0.9910	0.9534	0.9035	0.9617	0.40	19.33%

Table 3 - Dropout rate hyperparameter search with custom loss function

Dropout rate	AUROC	AUPRC	Macro-F1	Micro-F1	Threshold	Violation rate
0.0	0.9979	0.9922	0.9725	0.9886	0.40	8.93%
0.3	0.9945	0.9717	0.9282	0.9712	0.40	0.55%
0.4	0.9893	0.9481	0.8938	0.9511	0.35	0.44%
0.5	0.9825	0.9197	0.8600	0.9293	0.30	0.42%

Discussion

Architectural comparison: Width vs Depth

To justify the choice of a deeper GAT versus a wider GAT, Figures 5 and 6 are used as an example of a “wide” GAT (2 layers with 16 heads) and are contrasted against Figures 7 and 8, the final deeper GAT architecture (3 layers with 8 heads). Before discussing the results, it is important to consider a series of experimental variables to maintain a conservative view. Macro-F1 and Micro-F1 are reported but there was a change in design choice for their measurement later during development. In later stages, a search function to identify the best threshold to measure those metrics was introduced and the result is that the wide GAT Macro-F1 was calculated using a threshold of 0.5 (before the threshold search function was implemented) and the Macro- and MicroF1 scores for the deep GAT was 0.4 (after the implementation of an optimal threshold function). For this reason, it is better to maintain the focus on AUROC and AUPRC since their calculation was independent of the threshold. The other important consideration is that the wider GAT was only tested with the original λ of 0.5 (before discovering that the penalty loss was negligible at low λ) and therefore hierarchical violations (HV) were of around 20% vs 0.5% for the penalized, deeper model. In a way, the wider model was penalized for Macro-F1 by the absence of threshold search but advantaged by the absence of hierarchical penalty so results mathematically favor the wider model when it comes to AUROC and AUPRC.

For the loss curves, they are somewhat similar but it is important to remind ourselves that there is no hierarchical penalty applied to the wider model. This means that the wider model had similar losses without hierarchical penalties; a first step in showing the superiority of the deeper model. If we place our attention on the AUROC and AUPRC we can see a significant difference in performance in favor of the deeper model. For the AUROC, a difference of about 2% in performance may not seem significant but when we consider that the wider model was hierarchically unconstrained and the fact that the achieving higher performance when at the upper bound (near 0.99), the “deep” GAT comes out on top. For the AUPRC, a significant margin separates both models favoring the deeper model once more.

Custom loss function when unbound and hyper-constrained

When it was discovered that hierarchical penalties were insignificant at low λ , a plan was put in place to search for the ideal λ scheduling. Figure 9 shows the behavior of the loss curves when λ is not bound and allowed to increase infinitely without a ceiling for hierarchical violations. In this setting, initial lambda was 1 and increased by a factor of 1.2 every 5 epochs as long as there were hierarchical violations. The experiment ended with a dramatic increase in BCELoss and a λ exceeding 600. Under those conditions, the model will progressively focus on the hierarchical penalty as long as there are violations and will achieve a performance of 0.04% hierarchical violations (including the 0.1 margin mentioned earlier) but will at one point focus so much on hierarchical violations that it will forget to account for BCE losses. After a round of experimentations, it was determined empirically that an initial λ of 200 with a ceiling of 300 and a tolerance for hierarchical violations equal or below 1% was the best strategy to allow the model to balance both the BCE loss and the custom hierarchical penalty.

Dropout and convergence

The impact of dropout as a stabilizing regularizer is clearly evidenced by the contrast between Figures 12 and 13. In Figure 12 (Dropout = 0), the model exhibits a classic “overfitting pivot” at approximately epoch 80, where the validation loss begins a steady divergence from the training loss, ultimately triggering early stopping at epoch 135. This suggests that without dropout, the GAT-MLP layers began to memorize the specific protein pairs in the OhmNet training set rather than learning the generalized biological patterns of interaction.

Conversely, the introduction of a 30% dropout rate (Figure 13) significantly extended the model's "learning window". By forcing the network to develop redundant pathways for feature extraction, dropout delayed the onset of divergence, allowing for a more stable convergence that reached epoch 460 before triggering early stopping. This extended training duration was critical for the later integration of the hierarchical penalty, as it provided the model with a more stable weight initialization before the dynamic λ scheduling began to enforce ontological consistency.

Final results

The final results, presented in Table 3, represent the optimal balance between predictive power and biological logic. A critical observation was that while the model version without dropout achieved higher raw accuracy, it failed to maintain an acceptable Hierarchical Violation Rate (HVR > 8%). In contrast, models utilizing a 0.3 dropout rate maintained an HVR of approximately 0.5% across all trials.

This suggests a regularization synergy: dropout prevents the GAT layers from becoming overly 'rigid' or overfit to the training edgelist, thereby maintaining the parameter "flexibility" necessary for the hierarchical penalty to effectively steer the model toward ontological consistency. Without the noise introduced by dropout, a λ ceiling of 300 proved insufficient to redirect the model's attention; conversely, removing the ceiling would likely lead to the catastrophic BCE divergence observed in Figure 9. Thus, the combination of dropout and a capped dynamic λ is essential for stable, hierarchy-aware learning.

The Ohmnet framework

At the onset of the project, the primary goal was to surpass the original study's AUROC of 0.756 [1]. In spite of the differences in architecture and data preparation, the belief is that the goals have been exceeded with 0.995 as the final AUROC. Given such a significant performance gap, an analysis of the factors contributing to that improvement should be considered.

It is believed that the architectural differences are the main contributor to performance. By reaching beyond the traditional algorithmic regularization methods and adopting a late-fusion GAT-MLP model, this work was able to leverage the power of attention mechanisms for the model to attend to the most relevant protein interactions within a tissue-specific context. The ability for a GAT model to dynamically learn edge weights likely captured nuances that a static model could not.

While the original study likely omitted the 75 root tissues to optimize for computational overhead, this work demonstrates that including them, via label propagation, is essential for a neural network to develop a cohesive mathematical understanding of the human anatomy.

The use of 256d Node2Vec tissue embeddings also created a much richer representation space for the model to differentiate between tissues. The compatibility between the high-dimensional space and the GAT layers allowed the model to "triangulate" interactions to a degree of precision that was simply not possible in the 2017 study.

It is important to note that this high AUROC was achieved while simultaneously enforcing a strict Hierarchical Violation Rate (HVR) of less than 1%. This demonstrates that the GAT-MLP model does not need to sacrifice biological logic to achieve high predictive accuracy; rather, the biological hierarchy acts as a structural prior that may actually assist in refining the classification manifold, the underlying pattern or "shape" that the model creates in its "mind" to separate different categories of data.

In conclusion, while OhmNet established the foundational importance of hierarchy in PPI prediction, the integration of modern graph neural networks and dynamic penalty scheduling demonstrates that we can now achieve near-human-expert precision in tissue-specific interaction modeling.

Statement of Technical Implementation: This project was conceptualized and directed by the author. While the core biological logic, hierarchical constraints, and data auditing were original contributions, AI assistance was utilized for the technical implementation of the PyTorch Geometric framework and for vectorizing the custom hierarchical loss function. All architectural choices and hyperparameters were iteratively validated and tuned by the author to ensure scientific consistency. The notebook defines what is authored and what is assisted.

References

- [1] M. Zitnik and J. Leskovec, "Predicting multicellular function through multi-layer tissue networks," *Bioinformatics*, vol. 33, no. 14, pp. i190–i198, Jul. 2017, doi: 10.1093/bioinformatics/btx252.
- [2] "Early Fusion vs. Late Fusion in Multimodal Data Processing," GeeksforGeeks. Accessed: Mar. 05, 2026. [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/early-fusion-vs-late-fusion-in-multimodal-data-processing/>
- [3] G. Valentini, "True Path Rule Hierarchical Ensembles," Jun. 2009, pp. 232–241. doi: 10.1007/978-3-642-02326-2_24.
- [4] "read_edgelist — NetworkX 3.6.1 documentation." Accessed: Feb. 27, 2026. [Online]. Available: https://networkx.org/documentation/stable/reference/readwrite/generated/networkx.readwrite.edgelist.read_edgelist.html
- [5] "DiGraph—Directed graphs with self loops — NetworkX 3.6.1 documentation." Accessed: Feb. 27, 2026. [Online]. Available: <https://networkx.org/documentation/stable/reference/classes/digraph.html>
- [6] "Graph.nodes — NetworkX 3.6.1 documentation." Accessed: Feb. 27, 2026. [Online]. Available: <https://networkx.org/documentation/stable/reference/classes/generated/networkx.Graph.nodes.html>
- [7] "Tutorial — NetworkX 3.6.1 documentation." Accessed: Mar. 04, 2026. [Online]. Available: <https://networkx.org/documentation/stable/tutorial.html>
- [8] A. Grover and J. Leskovec, "node2vec: Scalable Feature Learning for Networks," Jul. 03, 2016, *arXiv*: arXiv:1607.00653. doi: 10.48550/arXiv.1607.00653.
- [9] "torch_geometric.nn.models.Node2Vec — pytorch_geometric documentation." Accessed: Mar. 01, 2026. [Online]. Available: https://pytorch-geometric.readthedocs.io/en/2.5.1/generated/torch_geometric.nn.models.Node2Vec.html
- [10] "BioSNAP: Network datasets: Tissue-specific protein-protein interaction network." Accessed: Mar. 05, 2026. [Online]. Available: <https://snap.stanford.edu/biodata/datasets/10013/10013-PPT-Ohmnet.html>
- [11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," Feb. 04, 2018, *arXiv*: arXiv:1710.10903. doi: 10.48550/arXiv.1710.10903.
- [12] "SparseAdam — PyTorch 2.10 documentation." Accessed: Mar. 05, 2026. [Online]. Available: <https://docs.pytorch.org/docs/stable/generated/torch.optim.SparseAdam.html>
- [13] O.-E. Ganea, G. Bécigneul, and T. Hofmann, "Hyperbolic Entailment Cones for Learning Hierarchical Embeddings," Jun. 06, 2018, *arXiv*: arXiv:1804.01882. doi: 10.48550/arXiv.1804.01882.
- [14] "FlexiDrop: Theoretical Insights and Practical Advances in Random Dropout Method on GNNs." Accessed: Mar. 02, 2026. [Online]. Available: <https://arxiv.org/html/2405.20012v1>
- [15] Google, "Protein Interaction Datasets and GPU Training," Gemini.

Appendix I – Supplemental Results